

~\OneDrive\Desktop\boolean-functions.py

```
1 import numpy as np
2 import random
3
4
5 class Perceptron:
6     def __init__(self, n_inputs, learning_rate=0.05):
7         self.n_inputs = n_inputs
8         self.learning_rate = learning_rate
9         # Initialize weights from normal distribution
10        self.weights = np.random.normal(0, 1/np.sqrt(n_inputs), n_inputs)
11        self.threshold = 0.0
12
13    def activation(self, x):
14        # Sign function: sgn(wx - threshold)
15        b = np.dot(self.weights, x) - self.threshold
16        return 1 if b > 0 else -1
17
18    def train(self, inputs, targets, epochs=20):
19        # Reset parameters for new training
20        self.weights = np.random.normal(0, 1/np.sqrt(self.n_inputs), self.n_inputs)
21        self.threshold = 0.0
22
23        for epoch in range(1, epochs + 1):
24            converged = True
25            # Train on all patterns
26            for i in range(len(inputs)):
27                x = inputs[i]
28                target = targets[i]
29                output = self.activation(x)
30
31                if output != target:
32                    converged = False
33                    # Update weights and threshold
34                    error = target - output
35                    self.weights += self.learning_rate * error * x
36                    self.threshold -= self.learning_rate * error
37
38            # Check if converged
39            if converged:
40                return True
41
42            # Not converged after all epochs
43            return False
44
45
46    def generate_input_patterns(n):
47        # Generate all possible n-bit patterns using counting
48        patterns = []
49        for i in range(2**n):
50            pattern = []
51            temp = i
```

```
52         for _ in range(n):
53             pattern.append(temp % 2)
54             temp = temp // 2 # used to shift right, a method that I learned with assembly
ARM32
55         patterns.append(np.array(pattern))
56     return patterns
57
58
59 def generate_all_boolean_functions(n):
60     # Generate all possible boolean functions
61     n_patterns = 2**n
62     n_functions = 2**n_patterns
63
64     functions = []
65     for i in range(n_functions):
66         function = []
67         temp = i
68         for _ in range(n_patterns):
69             # Convert 0/1 to -1/+1
70             function.append(1 if temp % 2 == 1 else -1)
71             temp = temp // 2
72         functions.append(function)
73     return functions
74
75
76 def test_all_functions(n):
77     # Test all boolean functions for n inputs
78     inputs = generate_input_patterns(n)
79     functions = generate_all_boolean_functions(n)
80
81     separable_count = 0
82     total_functions = len(functions)
83
84     for func_outputs in functions:
85         # tricky part: I had many errors in the start because using the same perceptron
instance
86         # caused weights to be updated from previous training and so the various
experiments were not independent
87         perceptron = Perceptron(n)
88         if perceptron.train(inputs, func_outputs):
89             separable_count += 1
90
91     fraction = separable_count / total_functions
92     return separable_count, total_functions, fraction
93
94
95 def sample_random_functions(n, n_samples=10000):
96     # Sample random boolean functions for large n
97     inputs = generate_input_patterns(n)
98     n_patterns = len(inputs)
99
100     separable_count = 0
101
102     for _ in range(n_samples):
```

```
103         # Generate random function
104         func_outputs = [random.choice([-1, 1]) for _ in range(n_patterns)]
105
106         perceptron = Perceptron(n)
107         if perceptron.train(inputs, func_outputs):
108             separable_count += 1
109
110     fraction = separable_count / n_samples
111     return separable_count, n_samples, fraction
112
113
114 def main():
115     print("Boolean Function Linear Separability Analysis")
116     print("=" * 50)
117
118     results = {}
119
120     # Test exhaustively for n=2,3
121     for n in [2, 3]:
122         print(f"\nTesting all functions for n={n}...")
123         separable, total, fraction = test_all_functions(n)
124         results[n] = fraction
125         print(f"n={n}: {separable}/{total} separable (fraction: {fraction:.4f})")
126
127     # Sample for n=4,5
128     for n in [4, 5]:
129         print(f"\nSampling 10000 functions for n={n}...")
130         separable, sampled, fraction = sample_random_functions(n, 10000)
131         results[n] = fraction
132         print(f"n={n}: {separable}/{sampled} separable (estimated: {fraction:.4f})")
133
134     # Analysis
135     print("\nResults show that the fraction of linearly separable")
136     print("Boolean functions decreases rapidly with n:")
137     for n in [2, 3, 4, 5]:
138         print(f"  n={n}: {results[n]*100:.2f}%")
139
140 if __name__ == "__main__":
141     main()
```